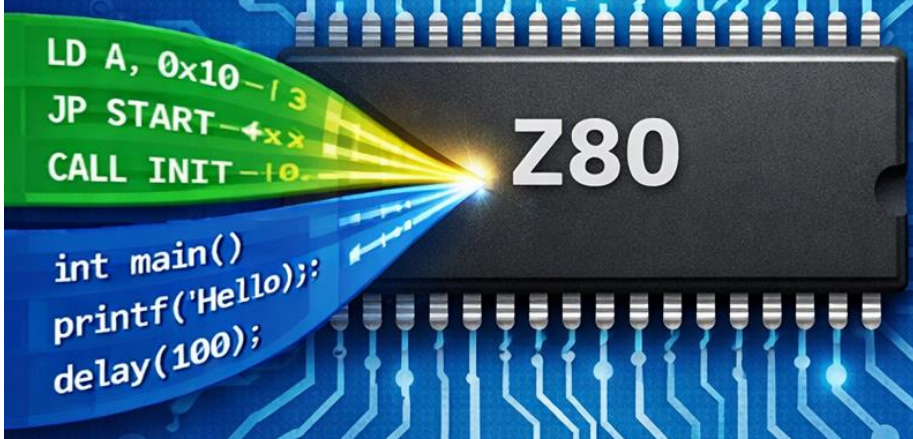
A Practical Guide to Z80 SBC Firmware Development with SDCC



Leonardo Chieco

A Practical Guide to Z80 SBC Firmware Development with SDCC

Several years ago, I wanted to design a Z80-based Single Board Computer (SBC) from scratch. While searching for reference designs, I discovered the well-known project by Grant Searle, which I still consider one of the most solid and well-thought-out Z80 SBC implementations available online.

The hardware design is simple, elegant, and robust, but what impressed me the most was the firmware: minimal, clear, and extremely well structured.
It follows a classic philosophy, making it easy to understand how the system boots, how interrupts are handled, and how serial I/O is implemented.

In recent days, I decided to revisit this idea and experiment with writing a complete Z80 firmware using the SDCC C compiler, combining C and assembly where appropriate. The objective was not performance at all costs, but clarity, control, and educational value.

Everything presented here is intended to be simple, explicit, and hackable, following the same philosophy that made the original Grant Searle design so effective.
Enjoy the reading!

The Author

Leonardo Chieco is an electronic engineer with over 20 years of experience in the design and development of automation control software (PC/PLC), electronic boards for industrial applications, firmware, robotics, and mechatronics.

LinkedIn: https://www.linkedin.com/in/leonardo-chieco-53550b129/

1. Introduction

This tutorial describes the design and implementation of bare-metal firmware for the Z80 CPU, intended for a minimal Single Board Computer (SBC) such as Grant Searle's, featuring:

- Zilog Z80 CPU
- Static RAM
- ROM or EEPROM
- MC6850 ACIA (or compatible) serial interface

The goal is not just to run a program, but to build a solid and extensible firmware base including:

- A serial monitor
- Proper interrupt-driven serial I/O
- A firmware that is understandable, hackable, and educational

No complex operating system, no BIOS, no hidden libraries but just Z80, C, and assembly.

2. Z80 SBC Hardware Architecture

CPU

The system is based on a Zilog Z80 CPU, providing a rich and well-structured architecture that made it extremely popular in early microcomputer designs:

- 16-bit addressing, allowing access to 64 KB of address space shared between RAM and ROM;
- 8-bit data bus with a powerful instruction set optimized for control-oriented applications;
- Multiple interrupt modes (IM 0, IM 1, IM 2), supporting both simple vectored interrupts and fully programmable interrupt handling;
- Dedicated I/O address space, accessed through the IN and OUT instructions, separate from memory addressing;
- Standardized reset and RST vectors, enabling fast jumps to fixed service routines;
- Multiple register sets (AF, BC, DE, HL and their shadow registers), allowing fast context switching in interrupt-driven systems;
- Index registers (IX, IY) for structured data access and table-based operations;
- Built-in refresh register (R), originally intended for DRAM refresh but also useful for timing and entropy sources;
- Flexible stack-based subroutine and interrupt handling, with automatic return address storage;
- Deterministic instruction timing, which simplifies real-time and low-level hardware control;

These features make the Z80 particularly well suited for small embedded systems, educational projects, and experimental firmware where full control over hardware and execution flow is required.

Interrupt Handling

The Z80 CPU supports three distinct interrupt modes, designed to cover a wide range of system complexities, from very simple designs to fully vectored and expandable interrupt architectures.

Interrupt Mode 0 (IM 0)

In IM 0, the interrupting device is expected to place an instruction directly on the data bus when an interrupt is acknowledged.
Typically, this instruction is a "`RST opcode`" or a "`CALL`" instruction.

This mode provides maximum flexibility but requires additional external hardware or glue logic to drive the data bus.
For this reason, IM 0 is rarely used in small SBC designs unless multiple interrupting devices must directly control execution flow.

Interrupt Mode 1 (IM 1)

IM 1 is the simplest and most commonly used interrupt mode in Z80 SBC systems.

When an interrupt occurs:

- the CPU automatically jumps to address 0x0038
- interrupts are disabled
- the return address is pushed onto the stack

The firmware must place an interrupt service routine (ISR) at address 0x0038, or a jump instruction redirecting execution elsewhere.

This mode is ideal for systems with:

- a single interrupt source (such as a serial interface),
- minimal hardware,
- and predictable interrupt behavior.

In this project, IM 1 is used to handle serial input interrupts generated by the ACIA, following the same approach used in Grant Searle systems.

Interrupt Mode 2 (IM 2)

IM 2 is the most powerful interrupt mode and allows for fully vectored interrupts.

In this mode:

- the CPU forms a 16-bit interrupt vector using the I register and a byte supplied by the interrupting device;
- this vector points to a table of ISR addresses in memory.

IM 2 enables multiple interrupt sources, clean separation of interrupt handlers, scalable and modular firmware designs. However, it requires more careful setup and is generally unnecessary for small systems with only one interrupt-generating peripheral.

For this firmware, IM 1 provides the best trade-off, keeping the design simple while still allowing efficient, interrupt-driven serial communication.

Without interrupts, serial I/O would rely on polling, constantly checking the ACIA status.

By using interrupts, the CPU continues to execute the code, and the ACIA intervenes only when necessary. This input architecture is reliable even at higher transmission speeds.
In IM 1, when an interrupt occurs, the Z80 always jumps to address 0038h. That address must contain a jump to the interrupt service routine (ISR).

Hardware Serial Interface – MC6850 ACIA

All user interaction with the system takes place through a Motorola MC6850 ACIA (Asynchronous Communications Interface Adapter).

The ACIA provides a simple and robust way to add asynchronous serial communication to 8-bit systems such as the Z80, handling most of the low-level UART details in hardware.

In this system, the MC6850 is memory-mapped in the Z80 I/O space as follows:

I/O Port	Function
0x80	Control / Status
0x81	Data

The ACIA acts as a bridge between the Z80 and the external serial line (typically RS-232 or TTL-level serial via a level shifter). It manages:

- Serial bit timing (baud rate, start/stop bits)
- Parallel-to-serial and serial-to-parallel conversion
- Interrupt generation on receive or transmit events

From the CPU point of view, interaction with the ACIA is extremely simple:
all communication happens through two I/O registers.

The MC6850 ACIA does not internally generate the baud rate.
The serial speed is entirely defined by an external clock source connected to the TxC/RxC pins.
For this reason, no baud rate configuration appears in the firmware.
This approach greatly simplifies the firmware and guarantees a stable serial timing independent of software execution.

The MC6850 ACIA is particularly well suited for small Z80 systems because:

- It requires only two I/O addresses
- It has minimal configuration complexity
- It supports interrupt-driven reception
- It integrates cleanly with Z80 IN/OUT instructions
- It works well with both ROM-based firmware and RAM-resident programs

Combined with a small software buffer and a simple ISR, it provides a reliable and efficient serial console for both the monitor and user applications.

Serial interrupt flow

Serial communication in this system is handled using an interrupt-driven approach, which allows the CPU to continue executing the main program without constantly polling the serial hardware.

When a character arrives on the serial line, the following sequence of events takes place.

First, the ACIA receives a character from the RS-232 interface and asserts its interrupt request (IRQ) line.

The Z80, operating in Interrupt Mode 1 (IM 1), immediately responds to this request. The CPU automatically pushes the current program counter onto the stack, preserving the execution state of the main program, then jumps to the fixed interrupt vector at address 0x0038.

At this location, the firmware contains a jump instruction that redirects execution to the C-level interrupt service routine, _serial_isr.

Inside the interrupt service routine:

- the ACIA status register is read to confirm that the interrupt was generated by a received character,
- the incoming byte is read from the ACIA data register,
- the character is stored into a circular buffer in RAM, allowing the system to decouple serial reception from application-level processing.
- Once the ISR has completed its work, it executes a RETI instruction, signaling the end of the interrupt service routine.

At this point, the Z80 restores the saved program counter and resumes execution exactly where the main program was interrupted, completely transparently.

This design ensures reliable serial input handling while keeping the main application code clean, responsive, and free from low-level hardware timing constraints.

Serial Driver Design

Incoming serial data is stored in a FIFO circular buffer in order to:

- Prevents data loss,
- Decouples ISR from application logic,
- Allows non-blocking input.

To prevent buffer overflow it is used RTS (Request To Send) signal as following:

- When the buffer is almost full → RTS HIGH
- When space is available again → RTS LOW

This closely follows Grant Searle's original firmware design.

Serial API
Application code never accesses the ACIA directly.
Instead, it uses a small API:

```
void serial_putc(uint8_t c);
uint8_t serial_getc(void);
uint8_t serial_char_available(void);
```

This makes the code cleaner, easier to reuse, independent of low-level hardware details.

Serial I/O Abstraction and Software Interrupts
In addition to the hardware interrupt used for receiving characters, the firmware also implements a set of software interrupt entry points to provide a clean and standardized interface for serial communication.

In the original Grant Searle monitor, these software interrupts are implemented using the Z80 `RST` instructions, which are single-byte opcodes that cause the CPU to jump to fixed addresses in low memory. This mechanism was widely used in classic Z80 systems such as NASCOM, CP/M machines, and many early monitors. I also followed this approach.

The following RST vectors are implemented:

- RST 08h – Output character
  Sends the character currently stored in register A to the serial port.

- RST 10h – Input character (blocking)
  Waits until a character is available in the software receive buffer, then returns it in register A.

- RST 18h – Check input availability
  Returns a non-zero value in register A if a character is available, zero otherwise.

Internally, these `RST` handlers call the same low-level serial routines used by the firmware itself. As a result, there is no duplication of logic and all serial I/O is handled consistently.

This approach provides several important advantages:

- It creates a stable ABI (Application Binary Interface) for serial I/O,
- Any program running on the system, whether written in C or assembly, can use serial services without knowing hardware details,
- Applications can be developed and loaded independently of the monitor or firmware,
- The system remains compatible with NASCOM-style and CP/M-style software conventions.

By combining hardware interrupts (for efficient and reliable reception) and software interrupts via RST instructions (for standardized I/O services), the

firmware achieves a clean separation between hardware handling, system services, and application-level code.

Memory Map

A typical memory map compatible with Grant Searle designs is:

Address Range	Size	Content
0000h – 1FFFh	8 kb	ROM Firmware / Monitor
2000h – FFFFh	56kb	RAM

Note: Some SBCs bank out the ROM after boot, but for clarity we assume a fixed ROM at low memory.

3. Project File Structure

crt0.s (C runtime)

- Reset vector
- Stack setup
- Interrupt vector
- Entry into C code

io.c / io.h

- Low-level Z80 I/O access
- C wrappers for IN / OUT instructions

serial.c / serial.h

- ACIA driver
- Interrupt Service Routine
- Circular buffer
- RTS flow control

monitor.c

- Command parser
- Interactive console

This firmware is not just a monitor, it is a **learning platform** for understanding how a real Z80 system works.

```
project_root/
│
├── src/
│   │
│   ├── crt0.s     ; Z80 startup code, reset vector, IM1 ISR vector, RST entry points
│   │
│   ├── io.h       ; Low-level I/O interface (IN / OUT)
│   ├── io.c       ; Z80 port I/O implementation using inline assembly
│   │
│   ├── serial.h   ; Serial driver interface (ACIA)
│   ├── serial.c   ; Interrupt-driven serial driver with circular buffer
│   │
│   ├── monitor.c  ; Command-line monitor (dump, poke, go, reset, etc.)
│   │
│   └── main.c     ; System entry point, initialization, monitor startup
│
├── build/
│   │
│   ├── crt0.rel   ; Assembled startup object
│   ├── io.rel     ; Compiled I/O module
│   ├── serial.rel ; Compiled serial driver
│   ├── monitor.rel; Compiled monitor
│   ├── main.rel   ; Compiled main module
│   │
│   ├── rom.ihx    ; Intel HEX output from SDCC linker
│   └── rom.bin    ; Final binary ROM image
│
└── Compile.bat    ; Build script
```

Let's see in detail what each software module does.

crt0.s – Startup Code and System Entry Points

The crt0.s file is the first code executed after reset. Its purpose is to bring the system into a known state and provide the fundamental entry points required by both the firmware and external programs.

At address 0x0000, the reset vector disables interrupts and jumps to the startup routine. This ensures that no interrupt can occur before the stack and memory are properly initialized.

The file also defines the IM 1 interrupt vector at address 0x0038, which immediately redirects execution to the serial interrupt service routine implemented in C (_serial_isr). This keeps the low-level interrupt handling minimal while allowing the main logic to be written in a high-level language.

The startup code initializes the stack pointer to a known RAM location, then calls the C main() function. From this point onward, execution proceeds entirely in C.

Finally, the file defines the software interrupt entry points using RST-compatible vectors:

- RST 08h – character output
- RST 10h – blocking character input
- RST 18h – input status check

These entry points form a stable interface that can be used by any application loaded into memory, independently of the monitor.

io.h / io.c – Low-Level I/O Access

The IO module provides a minimal abstraction layer for Z80 I/O port access.
Because the Z80 uses a dedicated I/O address space, port access must be performed using the IN and OUT instructions. These instructions are not directly expressible in standard C, so they are implemented using SDCC inline assembly. The functions:

- inb(port)
- outb(port, value)

are declared as __naked, which means that the compiler does not generate prologue or epilogue code, but only the assembler instructions of the function code. This allows precise control over registers and ensures the generated code maps directly to the intended Z80 instructions.

This module is intentionally small and generic, so it can be reused without modification in other Z80-based projects.

serial.h / serial.c – Interrupt-Driven Serial Driver

The serial module implements a fully interrupt-driven serial driver for the 6850 ACIA.
Its core responsibilities are:

- hardware initialization,
- interrupt handling,
- input buffering,
- and character-level I/O services.

Because the ACIA has only a one-byte hardware receive buffer, the interrupt service routine immediately reads incoming characters and stores them into a circular buffer in RAM. This prevents data loss and decouples serial reception from application timing.

Flow control is handled using RTS. This signal is asserted when the buffer approaches capacity, and released when sufficient space becomes available again.

The driver exposes a clean API:

- serial_getc(): blocking input
- serial_putc(): blocking output
- serial_char_available(): non-blocking input check

Internally, the ISR and the foreground code cooperate carefully, briefly disabling interrupts when shared state is modified to ensure consistency.

This design closely follows Grant Searle firmware conventions while remaining fully usable from C.

monitor.c – Command-Line Monitor

The monitor is the user-facing component of the firmware. It provides a simple command-line interface over the serial port, allowing direct interaction with system memory and execution flow.

After initialization, the monitor:

- prints a sign-on banner,
- displays a prompt,
- and enters an infinite command-processing loop.

Commands are intentionally simple and low-level, reflecting the role of the monitor as a development and diagnostic tool, not a full operating system.

Typical features include:

- memory dump and modification,
- memory search,
- execution of code at arbitrary addresses,
- system reset.

All input and output are performed using the serial abstraction, meaning the monitor itself is completely independent of the underlying hardware details.

4. Z80 SBC Monitor – User Manual

The monitor is a lightweight command-line interface running on our Z80 SBC.

All interaction happens via a serial terminal connected to the ACIA. Input is line-based, and all commands are executed after pressing Enter.

1. Sign-On and Prompt

When the monitor starts, it prints a banner and waits for input:

```
+--------------------+
| ZEOS MONITOR V1.0 |
+--------------------+
Ready
>
```

The > prompt indicates the system is ready to accept commands.
All commands are case-insensitive, but hexadecimal numbers must be written using digits 0-9 and letters A-F.

2. Available Commands

Memory Dump
D <addr> <len>

Displays memory contents in hexadecimal and ASCII format.
Parameters:

- addr – starting address (hexadecimal)
- len – number of bytes to display (hexadecimal)

The output is formatted as 8 bytes per line, with ASCII translation to the right. Non-printable characters are shown as “.”

Usage:
D 02F0 1F

```
  +--------------------+
  | ZEOS MONITOR V1.0 |
  +--------------------+

Ready
> D 02F0 1F
2D 2D 2D 2B 20 0D 0A 20     ---+ ..
20 7C 20 5A 45 4F 53 20      | ZEOS
4D 4F 4E 49 54 4F 52 20     MONITOR
56 31 2E 30 20 7C 20        V1.0 |
Ready
>
```

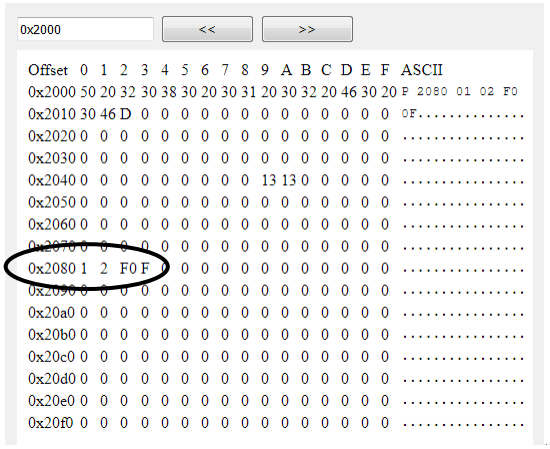
Poke Memory
P <addr> <v..>

Writes bytes to memory. Useful for testing or modifying program data.
Parameters:

- addr – starting address (hexadecimal)
- v.. – space-separated values in hexadecimal

The monitor checks for memory overflow before writing. If the command is invalid, it reports Invalid input.

Usage:
P 2080 01 02 F0 0F




```
.
.   +--------------------+
.   | ZEOS MONITOR V1.0 |
.   +--------------------+
.
.Ready
.> P 2080 01 02 F0 0F
.Done
.Ready
.>
```

Fill Memory (RAM)

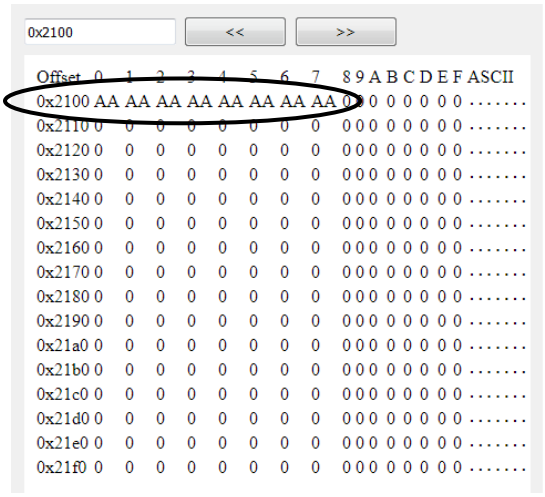
F <addr> <len> <val>

Fills a memory range with a single byte value.
Parameters:

- addr – starting address (hexadecimal)
- len – how many times to write the value
- val – value to write

Usage:

`F 2100 08 AA`




```
   +--------------------+
   | ZEOS MONITOR V1.0 |
   +--------------------+

Ready
> F 2100 08 AA
Done
Ready
```

Go - Jump to Address

G <addr>

Executes code at the specified address. This is equivalent to calling a function pointer in C.
Parameter:

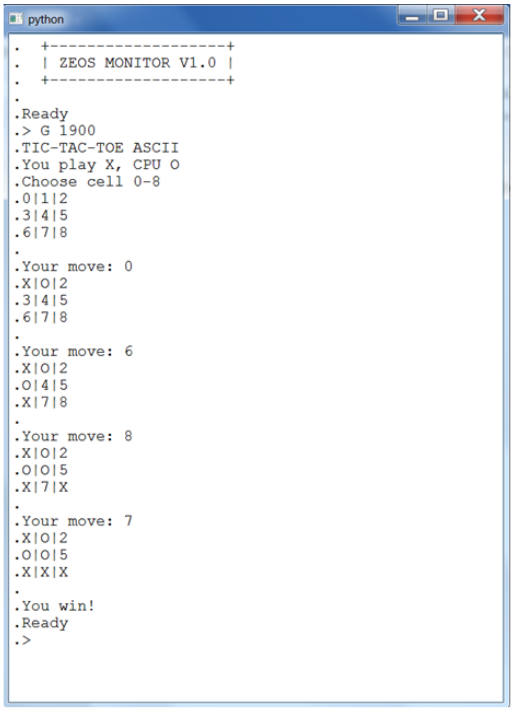
- addr – target address to jump to (hexadecimal)

Can be used to launch standalone programs at a known memory location.

After executing, the program takes full control.
The monitor will not regain control until the program returns (if ever).
Usage:

```
G 1900
```

During ROM compilation, two programs were inserted at addresses 0x1800 and 0x1900. The first, written in assembly, prints a message; the second, written in C, implements the "tic tac toe" game.



Memory Pattern Search
M <start> <end> <p..>

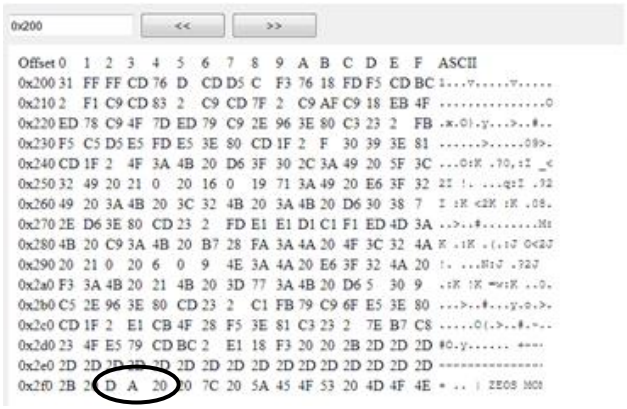
Searches memory for a specific pattern of bytes and prints all addresses where the pattern occurs.

Parameters:

- start – start address (hexadecimal)
- end – end address (hexadecimal)
- p.. – byte pattern to search for

Usage:

```
M 1000 1FFF 12 34 56
```



X – System Reset

Resets the CPU and restarts the system, equivalent to a cold start.

Usage:
```
X
```

All memory is unchanged unless explicitly cleared by the program.

HELP – Display Command List

Displays a summary of all available commands and their syntax.

This is useful when using the monitor for the first time or when experimenting with new memory locations.

Usage:
H

```
  +--------------------+
  | ZEOS MONITOR V1.0 |
  +--------------------+

Ready
> H
Commands:
D <addr> <len>       : Dump memory in hex + ASCII
P <addr> <v..>       : Poke memory bytes
F <addr> <len> <val> : Fill memory with a value
G <addr>             : Jump to address (go)
M <start> <end> <p..>: Search memory pattern
X                    : Reset system
H                    : Show this help
Ready
>
```

Important command Notes

Hexadecimal input: All addresses, lengths, and values must be provided in hexadecimal.

Memory safety: The monitor performs basic checks to prevent writing beyond the 64 KB memory space.

Blocking input/output: All serial I/O is handled using the interrupt-driven driver, so commands respond in real time without losing characters.

Integration with software interrupts: Programs can use RST 08h / 10h / 18h to interact with the serial port independently of the monitor.

This manual provides the foundation for interacting with the monitor, performing low-level testing, launching standalone programs, or experimenting with assembly/C code on the Z80 SBC.

5. Conclusions

This work is not intended to be a complete or definitive firmware, but rather a starting point for anyone who, even in 2026, wants to explore and experiment with the Z80 ecosystem.

The Zilog Z80 is a revolutionary and remarkably long-lived microprocessor. Despite its age, it still offers an extraordinary learning platform: a clean architecture, a simple yet powerful instruction set, well-defined hardware interfaces, and full transparency between hardware and software. These characteristics make it ideal for understanding how a computer truly works at its lowest levels.

This tutorial demonstrates that it is still possible, and extremely rewarding, to build modern, structured firmware using a historical CPU, combining low-level assembly with C code through the SDCC toolchain. Starting from a minimal runtime, interrupt-driven serial I/O, and a simple monitor, we showed how to design a reusable software foundation that can be extended with new applications, games, and experiments.
Learning to program the microprocessors that shaped the history of computing is more than an exercise in nostalgia. It provides a solid conceptual foundation: understanding interrupts, memory maps, I/O, timing, and resource constraints builds skills that are directly transferable to modern embedded systems and even to more complex software architectures.

Special thanks go to Grant Searle (http://www.searle.wales/) for his excellent Z80 SBC design and for the clarity and simplicity of the original firmware, which remains a reference even today. His work has enabled countless enthusiasts to approach Z80 hardware in a practical and accessible way.
Thanks also to cburbridge (https://github.com/cburbridge/z80) for developing a Grant Searle SBC emulator, a simple tool for testing, debugging, and learning without requiring physical hardware.